

DeepAssessAI 技术文档

项目成员：10235501431 曹欣元，10235501420 刘耀辉

实验环境配置：

1.软件基础环境

操作系统：Windows 10/11 或 macOS。

前端运行时：Node.js v18.0.0+，用于驱动 Electron 桌面框架。

后端语言：Python 3.9+，利用其强大的 AI 生态库和异步框架。

数据库：Redis 6.0+，用于高性能数据缓存与错题持久化。

2.云原生部署配置 项目使用 Docker Compose 进行环境编排。backend 服务基于 Python 镜像构建，并映射 8000 端口；redis 服务使用镜像自带的数据卷挂载技术，将容器内的 /data 目录映射到宿主机 redis_data 卷，确保数据持久性。

3.关键依赖库

后端：FastAPI (Web 框架)、PyMuPDF (PDF 解析)、OpenAI (LLM 接入)、Redis (驱动)。

前端：Electron (桌面容器)、Axios (网络请求)、Electron-Store (本地配置存储)。

一、 系统逻辑架构概览

本项目采用典型的桌面客户端与云端服务分离的架构，确保了本地操作的流畅性与云端计算的高效性。

1.1 桌面客户端逻辑 (Electron)

客户端不仅是 UI 容器，更承担了原生系统调用的职责。主进程负责窗口控制与文件对话框唤起，渲染进程负责界面交互，预加载脚本 (Preload) 作为安全桥梁，允许前端以受限方式调用 Node.js 的文件读取能力。

1.2 后端服务逻辑 (FastAPI)

后端是系统的逻辑中心，负责处理文件流解析、用户权限管理、大模型（LLM）的 Prompt 构建以及基于 Redis 的数据持久化调度。

1.3 数据交互模型

系统基于 RESTful API 协议，数据交换统一采用 JSON 格式。后端通过异步协程池处理高耗时的 AI 生成任务，确保前端界面不卡顿。

二、 用户认证与会话管理逻辑

2.1 注册与登录的前端逻辑

数据捕获：前端 JS 监听表单提交事件，实时捕获用户输入的账号与加密后的密码。

异步验证：通过 Axios 向后端发起 POST 请求。如果后端返回成功，前端将用户信息序列化后存储至浏览器的 LocalStorage 中。

路由保护：系统会检查本地存储中的用户凭证，若凭证缺失或过期，逻辑上会自动拦截非登录页面的访问。

2.2 账号校验的后端逻辑

注册逻辑：后端接收请求后，首先查询 Redis 键空间。若用户名已存在，则抛出 400 错误；若不存在，则创建对应的 Hash 记录。

登录逻辑：从 Redis 中提取存储的凭证进行比对。校验通过后，后端会更新用户的“最后登录时间”等元数据。

```
@app.post("/api/register")
def register(user: UserAuth):
    if r.exists("users", user.username):
        raise HTTPException(status_code=400, detail="用户已存在")
    r.hset("users", user.username, user.password)
    return {"status": "success"}

@app.post("/api/login")
def login(user: UserAuth):
    db_pass = r.hget("users", user.username)
    if db_pass == user.password:
        return {"status": "success", "username": user.username}
```

```
raise HTTPException(status_code=401, detail="密码错误")
```

三、 后端接口实现细节全解析

3.1 资料解析接口 (/api/upload_to_cache)

流程：后端接收前端通过二进制流传输的文件。

细节：如果是 PDF，后端启动 PyMuPDF 扫描页面对象并提取纯文本。如果是文本文件，后端执行“编码自动探测逻辑”，先尝试 UTF-8，若失败则切换至 GBK，彻底解决乱码问题。

缓存：解析出的文本会被存入 Redis 缓存区，并赋予一个长效 UUID，确保用户在同一会话内无需重复解析。

```
@app.post("/api/upload_to_cache")
async def upload_to_cache(username: str, file: UploadFile = File(...)):
    """上传文件：解析、以文件名建类、标记为最近上传"""
    content = await file.read()
    if file.filename.lower().endswith(".pdf"):
        text = utils.extract_text_from_pdf_bytes(content)
    else:
        text = utils.extract_text_from_txt_bytes(content)

    if not text:
        raise HTTPException(status_code=400, detail="解析失败")

    set_id = str(uuid.uuid4())
    set_meta = {
        "id": set_id,
        "name": file.filename,
        "created_at": str(uuid.uuid1().time)
    }
```

```
    # 存储分类信息
    r.hset(f"sets:{username}", set_id, json.dumps(set_meta,
ensure_ascii=False))
    r.set(f"text_content:{set_id}", text)
    # 标记为该用户的“当前活跃分类”
    r.set(f"recent_set:{username}", set_id)
```

```
    return {"status": "success", "set_id": set_id, "category_name":
file.filename}
```

3.2 批量题目生成接口 (/api/generate_batch)

配置解析：后端接收包含题型、数量、难度的复杂字典。

并发调度：这是系统的核心难点。后端不再逐一调用 AI，而是将出题需求拆分为多个独立的协程任务，利用 Python 异步机制并发请求智谱 AI 接口，将原本需要数分钟的生成过程压缩至十几秒。

格式保护：Agent 在生成过程中，后端会强制要求其输出结构化 JSON。

```
@app.post("/api/generate_batch")
async def generate_batch(req: BatchGenerateRequest):
    """自动寻找最近文档并按嵌套配置出题"""
    # 自动识别 ID
    target_id = r.get(f"recent_set:{req.username}")
    if not target_id:
        raise HTTPException(status_code=404, detail="请先上传文档")

    text = r.get(f"text_content:{target_id}")
    tasks = []

    # 解析嵌套 JSON
    for q_type, diff_config in req.config.items():
        for diff_level, count in diff_config.items():
            difficulty = int(diff_level)
            if q_type == "mcq":
                for _ in range(count):
                    tasks.append(asyncio.to_thread(utils.generate_one_mcq, text, difficulty))
            elif q_type == "short":
                for _ in range(count):
                    tasks.append(asyncio.to_thread(utils.generate_one_short, text, difficulty))

    results = await asyncio.gather(*tasks)

    final_questions = []
    for q in results:
        if q:
            # 存入历史库
            r.rpush(f"questions:{target_id}", json.dumps(q, ensure_ascii=False))
            final_questions.append(q)
```

```
    return {"status": "success", "set_id": target_id, "questions":  
final_questions}
```

3.3 智能判卷接口 (/api/grade_short 和/api/save_mistake)

语义对比：后端将用户回答、参考答案、题目背景一并打包发给 AI 老师。

评分逻辑：AI 根据关键知识点匹配度给出 0-100 的分值。

自动归档：后端在此处设计了一个“自动错题拦截器”。若评分低于 60 分，系统会在返回响应前，直接将该题详情写入 Redis 的错题列表（List）中。

```
@app.post("/api/save_mistake")  
def save_mistake(req: MistakeRequest):  
    """选择题选错：实时给分析并入库"""  
    target_id = req.set_id or r.get(f"recent_set:{req.username}")  
  
    ai_feedback = utils.get_mistake_feedback(  
        req.question.get('question'),  
        req.question.get('reference_answer'),  
        req.user_answer  
    )  
  
    record = {"q": req.question, "my_ans": req.user_answer, "ai_feedback":  
ai_feedback}  
    r.lpush(f"mistakes:{target_id}", json.dumps(record,  
ensure_ascii=False))  
  
    return {"ai_feedback": ai_feedback, "correct":  
req.question.get('reference_answer')}  
  
@app.post("/api/grade_short")  
def grade_short(req: GradeRequest):  
    """简答题阅卷：不及格（<60）自动入库"""  
    result = utils.grade_short_answer(  
        req.question_data.get('question'),  
        req.question_data.get('reference_answer'),  
        req.user_answer  
    )  
  
    score = result.get('score', 0)  
    comment = result.get('comment', "无评语")
```

```

    if score < 60:
        target_id = req.set_id or r.get(f"recent_set:{req.username}")
        record = {"type": "short", "q": req.question_data, "my_ans":
req.user_answer, "ai_feedback": comment}
        r.lpush(f"mistakes:{target_id}", json.dumps(record,
ensure_ascii=False))

    return {"score": score, "comment": comment}

```

3.4 历史与错题查询接口 (/api/sets/list, /api/mistakes/list)

资料库检索：通过 Redis 的 HGETALL 命令，快速拉取当前用户下的所有资料元数据。

错题本提取：利用 Redis 的 LRANGE 命令，按时间倒序提取用户的错题记录，确保用户优先看到最新的错题。

```

@app.get("/api/sets/list")
def get_user_sets(username: str):
    data = r.hgetall(f"sets:{username}")
    result = []
    for sid, meta_json in data.items():
        meta = json.loads(meta_json)
        meta['count'] = r.llen(f"questions:{sid}")
        result.append(meta)
    return sorted(result, key=lambda x: x['created_at'], reverse=True)

```

```

@app.get("/api/sets/detail")
def get_set_questions(set_id: str):
    items = r.lrange(f"questions:{set_id}", 0, -1)
    return [json.loads(i) for i in items]

```

```

@app.get("/api/mistakes/list")
def get_set_mistakes(set_id: str):
    items = r.lrange(f"mistakes:{set_id}", 0, -1)
    return [json.loads(i) for i in items]

```

四、 前后端交互流程详解

4.1 桌面端原生文件读取流程

交互触发：用户点击“导入文件”。

原生调用：Electron 主进程通过系统 API 弹出文件选择器，并将选中的绝对路径返回给渲染进程。

预加载读取：渲染进程通过预加载脚本调用 `fs.readFileSync` 将文件转为 Buffer，通过 Axios 以表单形式上传给后端。

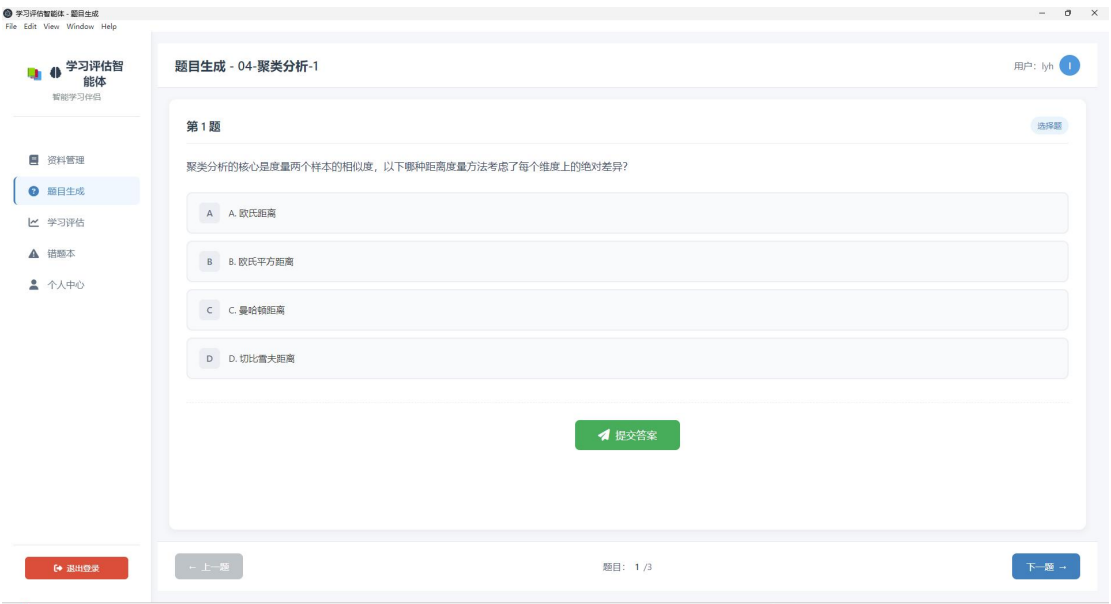
4.2 刷题状态同步流程

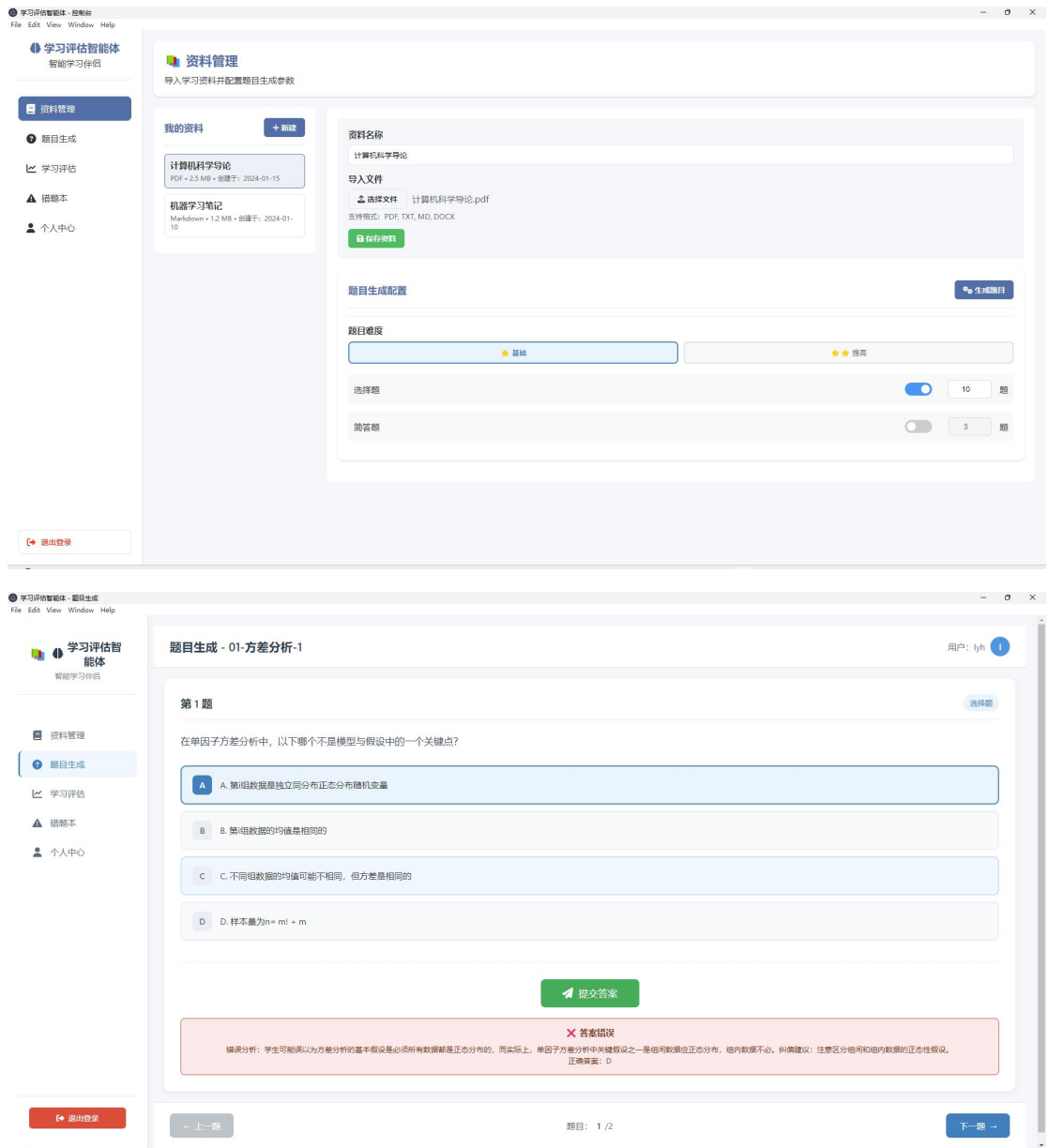
题目分发：前端请求生成题目后，后端返回一个 JSON 数组。前端将其存入当前页面的内存状态机中。

实时同步：用户切换题目或输入答案时，前端 JS 实时更新本地数组。

最终汇总：用户点击“提交整卷”时，前端将更新后的整个数组发送给后端进行批量归档。

4.3 前端页面展示





五、 LLM Agent 智能设计细节

5.1 难度梯度控制逻辑

后端 `utils.py` 实现了一套动态 Prompt 生成器。

低难度：Prompt 明确限制 AI 只能提取文档原文。

高难度：Prompt 要求 AI 基于文档内容构造“反直觉”的干扰项，或要求用户在多个正确表述中选择“最合适”的一项。

5.2 解析鲁棒性逻辑

为了防止 AI 输出的无关文字（如“好的，这是为您生成的题目”）破坏前端逻辑，后端实

现了一个“JSON 锚点提取器”。该逻辑会扫描 AI 返回的全文，精准定位最外层的花括号，忽略所有非 JSON 字符，保证了前后端通讯的稳定性。

六、 实验分工说明

刘耀辉：负责 Electron 主进程逻辑、Preload 安全通信、前端登录状态持久化、以及 Axios 全局请求拦截器的封装。

曹欣元：负责 FastAPI 后端接口逻辑、Redis 错题库存储设计、PDF 文本解析引擎开发、以及大模型 Agent 提示词的高级调优。